Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. **Understand Asymptotic Notation:**
    - Explain Big O notation and how it helps in analyzing algorithms.
    - Describe the best, average, and worst-case scenarios for search operations.
2. **Setup:**
    - Create a class **Product** with attributes for searching, such as **productId, productName**, and **category**.
3. **Implementation:**
    - Implement linear search and binary search algorithms.
    - Store products in an array for linear search and a sorted array for binary search.
4. **Analysis:**
    - Compare the time complexity of linear and binary search algorithms.
    - Discuss which algorithm is more suitable for your platform and why.

SOLUTION:

Big O Notation:

Big O notation is used to describe the performance or efficiency of an algorithm as the input size (**n**) grows.

It helps us:

- Measure algorithm efficiency.
- Compare different algorithms.
- Predict performance for large datasets.

Examples:

- **O(1)** → Constant time

- **O(log n)** → Logarithmic time
- **O(n)** → Linear time
- **O(n^2)** → Quadratic time

Search Operation Cases:

Case	Description
Best Case	Element found at the first position
Average Case	Element found somewhere in the middle
Worst Case	Element found at the last position or not found

Linear Search

- Best Case: **O(1)**
- Average Case: **O(n)**
- Worst Case: **O(n)**

Binary Search

- Best Case: **O(1)**
- Average Case: **O(log n)**
- Worst Case: **O(log n)**

Coding:

```
class Product {
  int productId;
  String productName;
  String category;

  Product(int productId, String productName, String category) {
    this.productId = productId;
    this.productName = productName;
```

```
        this.category = category;
    }

    @Override
    public String toString() {
        return "ID: " + productId +
               ", Name: " + productName +
               ", Category: " + category;
    }
}
class LinearSearch {

    public static Product search(Product[] products, int targetId) {

        for (Product p : products) {
            if (p.productId == targetId) {
                return p;
            }
        }

        return null;
    }
}

public class BinarySearch {
        public static Product search(Product [] products,int targetId) {
                int l=0;
                int r=products.length;
```

```
        while(l<=r) {
            int mid=l+(r-l)/2;
            if(products[mid].productId==targetId) {
                return products[mid];
            }
            else if(products[mid].productId<targetId) {
                l=mid+1;
            }
            else {
                r=mid-1;
            }
        }
        return null;
    }
}
public class EcommerceSearch {

  public static void main(String[] args) {

    Product[] products = {
      new Product(101, "Laptop", "Electronics"),
      new Product(102, "Mouse", "Electronics"),
      new Product(103, "Keyboard", "Electronics"),
      new Product(104, "Chair", "Furniture"),
      new Product(105, "Table", "Furniture")
    };

    int targetId = 104;
```

```
        Product linearResult =
            LinearSearch.search(products, targetId);

        System.out.println("Linear Search Result:");
        System.out.println(linearResult);

        Product binaryResult =
            BinarySearch.search(products, targetId);

        System.out.println("\nBinary Search Result:");
        System.out.println(binaryResult);
    }
}
```

Output:

```
<terminated> EcommerceSearch [Java Application] C:\Users\Asus\.p2\pool\plugins\org.
Linear Search Result:
ID: 104, Name: Chair, Category: Furniture

Binary Search Result:
ID: 104, Name: Chair, Category: Furniture
```

Analysis:

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)

Binary Search	O(1)	O(log n)	O(log n)

Example

Suppose there are 1,000,000 products**:**

Linear Search

- May need up to 1,000,000 comparisons.

Binary Search

- Needs approximately:

$$\text{Log}_2(1000000) \approx 20$$

comparisons only.

<u>Which Algorithm is More Suitable?</u>

Binary Search is More Suitable

Reasons:

1. Much faster for large product catalogs.
2. Time complexity is O(log n)**.**
3. Reduces server response time.
4. Improves user experience by returning search results quickly.

Limitation

Binary Search requires:

- Data to be sorted by the search key (e.g., productId).
- Additional maintenance when inserting new products.

Conclusion

For an e-commerce platform with thousands or millions of products, Binary Search is the preferred choice because it provides significantly faster search performance than Linear Search. Linear Search is simple to implement but becomes inefficient as the number of products grows.

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**
    - **Explain the concept of recursion and how it can simplify certain problems.**
2. **Setup:**
    - **Create a method to calculate the future value using a recursive approach.**
3. **Implementation:**
    - **Implement a recursive algorithm to predict future values based on past growth rates.**
4. **Analysis:**
    - **Discuss the time complexity of your recursive algorithm.**
    - **Explain how to optimize the recursive solution to avoid excessive computation.**

SOLUTION:

Understanding Recursive Algorithms

What is Recursion?

Recursion is a programming technique where a method calls itself to solve a smaller version of the same problem until a base condition is reached.

A recursive solution consists of:

1. **Base Case – Stops the recursion.**
2. **Recursive Case – Calls the function with a smaller problem.**

Why Use Recursion?

- **Makes code shorter and easier to understand for problems that have repetitive structures.**
- **Naturally models mathematical formulas and forecasting calculations.**

- **Useful for divide-and-conquer and sequence-based computations.**

Setup

Assume:

- **Current value = P**
- **Growth rate = r**
- **Number of years = n**

Future Value Formula:

$FV=P\times(1+r)^n$

Using recursion:

$FV(n)=FV(n-1)\times(1+r)$

Base Case:

$FV(0)=P$

CODING:

```
public class FinancialForecast {

  public static double futureValue(double currentValue,
                                   double growthRate,
                                   int years) {

    if (years == 0) {
      return currentValue;
    }

    return futureValue(currentValue, growthRate, years - 1)
        * (1 + growthRate);
  }
```

```
    public static void main(String[] args) {

        double currentValue = 10000;
        double growthRate = 0.08;
        int years = 5;

        double result =
            futureValue(currentValue, growthRate, years);

        System.out.printf(
            "Predicted value after %d years: %.2f",
            years, result);
    }
}
```

OUTPUT:

```
<terminated> FinancialForecast [Java Application] C:\Users\Asus\.p2\p
Predicted value after 5 years: 14693.28
```

Analysis

Time Complexity

For the recursive algorithm:

futureValue(value, rate, n)

The function performs one recursive call for each year.

Recurrence relation:

$T(n)=T(n-1)+O(1)$

Therefore:

T(n)=O(n)

Space Complexity

Since each recursive call is stored on the call stack: O(n)

<u>Optimizing the Recursive Solution</u>

Problem with Simple Recursion

For large values of n, recursion:

- **Uses more memory due to the call stack.**
- **May cause stack overflow.**
- **Performs unnecessary function-call overhead.**

Optimization 1: Iterative Approach

```
public class FinancialForecast {

        public static double futureValueIterative(
            double currentValue,
            double growthRate,
            int years) {

          double result = currentValue;

          for (int i = 0; i < years; i++) {
            result *= (1 + growthRate);
          }

          return result;
        }


  public static void main(String[] args) {
```

```
        double currentValue = 10000;
        double growthRate = 0.08;
        int years = 5;

        double result =
                futureValueIterative(currentValue, growthRate, years);

        System.out.printf(
            "Predicted value after %d years: %.2f",
            years, result);
    }
}
```

```
<terminated> FinancialForecast [Java Application] C:\Users\Asu
Predicted value after 5 years: 14693.28
```

- **Time Complexity: O(n)**
- **Space Complexity: O(1)**

Optimization 2: Fast Exponentiation

Using:

$FV = P \times (1+r)^n$

Compute $(1+r)^n$ using exponentiation by squaring:

```
public class FinancialForecast {

        public static double power(double base, int exp) {
          if (exp == 0)
             return 1;
```

```
        double half = power(base, exp / 2);

        if (exp % 2 == 0)
            return half * half;
        else
            return base * half * half;
    }

    public static double futureValueOptimized(
        double currentValue,
        double growthRate,
        int years) {

        return currentValue *
            power(1 + growthRate, years);
    }


public static void main(String[] args) {

    double currentValue = 10000;
    double growthRate = 0.08;
    int years = 5;

    double result =
            futureValueOptimized(currentValue, growthRate, years);
```

```
        System.out.printf(
            "Predicted value after %d years: %.2f",
            years, result);
    }
}
```

```
<terminated> FinancialForecast [Java Application] C:\Users\Asus\.p2\pool\plugins\
Predicted value after 5 years: 14693.28
```

Optimized Complexity

Method	Time Complexity	Space Complexity
Simple Recursion	O(n)	O(n)
Iterative	O(n)	O(1)
Fast Exponentiation	O(log n)	O(log n)